B-Trees

Design and Analysis of Algorithms

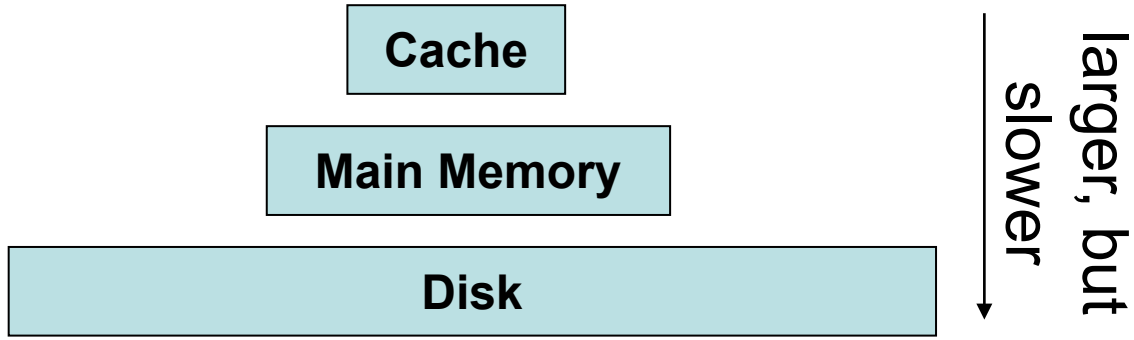
Brian C. Dean

School of Computing

Clemson University

Motivation: Block Memory Transfers

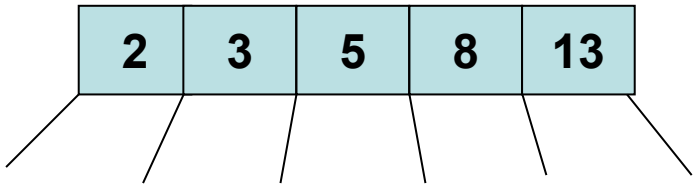
- Most memories are hierarchical in structure:



- Between levels, data often transferred in blocks (say, 1K at a time).
- Often the true performance of a data structure is determined by the number of blocks it accesses.
- How many disk accesses might be performed by a balanced binary search tree holding 1 billion records?

The B-Tree : Structure

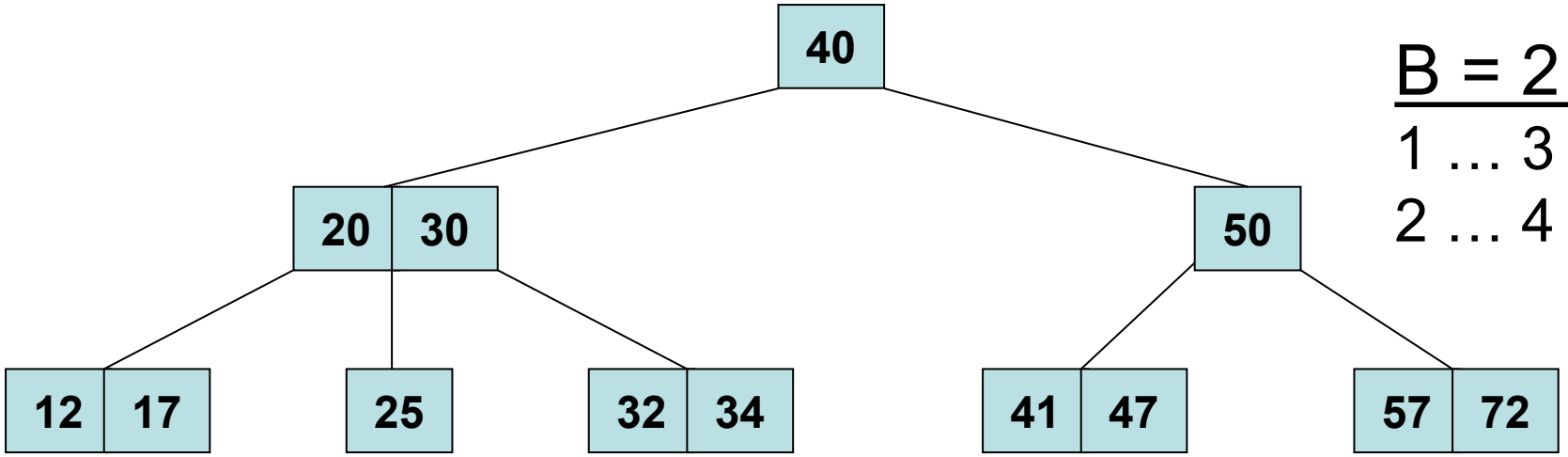
- Each node in a binary tree has 1 key and 2 children (some children possibly NULL)
- In a B-tree, each node stores B – 1 … 2B – 1 keys and therefore has B … 2B children (keys / children usually stored in arrays):



- The root is special, having no lower limit. It could store a single key.
- B typically chosen to align with block transfer size (e.g., from disk to main memory). Commonly B quite large; e.g., B = 1000.

The B-Tree : Structure

- All leaves have the same depth, $O(\log_B n)$.

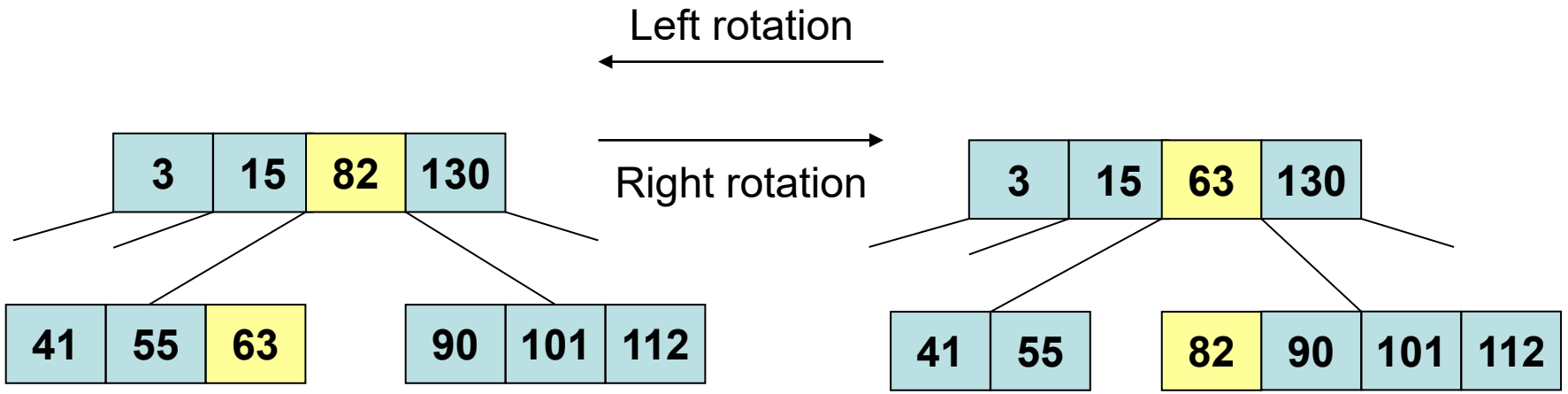


B = 2 (a "2-3-4 tree"):
1 … 3 keys per node
2 … 4 children per node

- All operations on a B-tree (e.g., *insert, delete, find, pred*, *succ*, *min*, *max, rank, select*) can be easily implemented in $O(B \log_B n)$ time.
- The "$\log_B n$" term is the most important, related to block transfers. If n = 1 billion and B = 1000, then $\log_B n = 3$ (versus ~30 for a BST).

Sharing With Your Siblings

- Rotations allow us to donate or steal elements from a sibling.



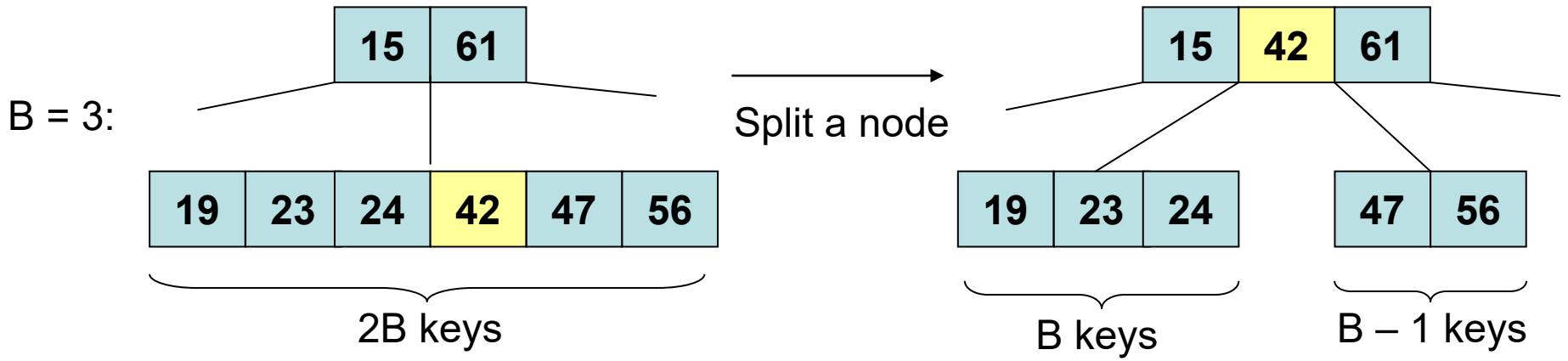
- A rotation can be implemented in O(B) time

Insertion and Deletion

- These always take place in leaf nodes.
- To ensure we delete from a leaf node, we may need to swap first with our predecessor or successor (remember the “ugly” method for deleting a node w/2 children from a BST).
- The concern: insertion might make a node too large, or deletion might make a node too small.
  - Can sometimes fix by sharing with a sibling via rotation.

Splitting a Node after Insertion

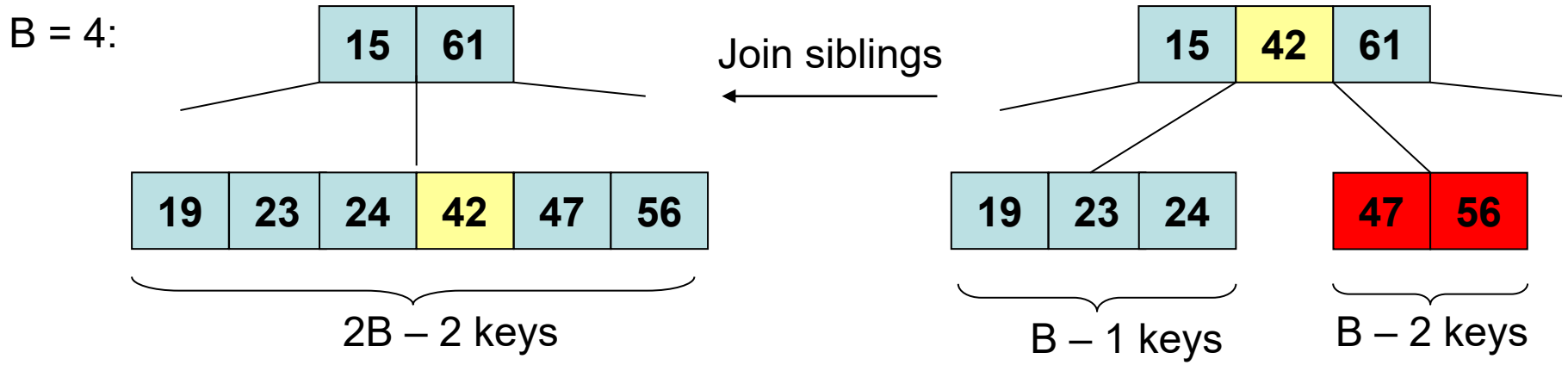
- Insertion might give a node too many keys (2B of them).
- If so, we can split the node and donate its median element to the parent:



- This might give the parent too many elements, causing it to split, etc.
- Splitting all the way to the root is the only way a B-tree increases in height.

Joining Two Nodes After Deletion

- Deletion might give a node too few keys (B – 2 of them).
- Try to fix this by stealing from a sibling.
- If this fails, we steal 1 element from our parent and join with an adjacent sibling.



- This might give the parent too few elements, causing it to join with a sibling, etc.
- Joining all the way to the root is the only way a B-tree decreases in height.

Recall: "Equivalence" with Red-Black Trees

- Interesting bit of trivia – a "2-3-4" tree (B=2) is essentially equivalent to a red-black tree!